

SymPlan: Enhancing Neurosymbolic Mathematical Reasoning in Compact Language Models via Representation and Planning

Tay Bui^{1,2}, Huy N. G. Pham^{1,2}, and Nhan Ha Le Thanh³

¹ University of Information Technology, Ho Chi Minh City, Vietnam

² Vietnam National University, Ho Chi Minh City, Vietnam

³ FPT University, Ho Chi Minh City, Vietnam

{24521589, 24520690}@gm.uit.edu.vn, hnhan5355@gmail.com

Abstract. Executable symbolic programming helps language models overcome calculation errors in mathematical reasoning. While frontier models succeed by jointly interpreting, planning, and generating code in a single unconstrained pass, compact small language models (SLMs, $\leq 8B$) under 4-bit quantization collapse under this monolithic burden. Limited capacity induces cognitive overload, causing semantic goal drift, omitted domain boundaries, and repair regressions. To bridge this gap without parameter scaling, we propose **SymPlan**, an inspectable neurosymbolic framework for resource-constrained reasoning. SymPlan establishes an immutable *target anchor* that locks the queried unknown and its formal output signature prior to code generation, stabilizing SymPy synthesis and preventing regressive overwrites during debugging. Across six open-weight quantized models (3B–8B) on MATH-500, SymPlan outperforms monolithic SymCode by +3.60% to +5.20% (averaging +4.20%) and Chain-of-Thought by +12.64%, raising execution success to 89.90% without fine-tuning.

Keywords: Neurosymbolic AI · Small Language Models · Mathematical Reasoning · SymPy Code Generation · Algorithmic Planning.

1 Introduction

Chain-of-Thought (CoT) prompting [15] and structured search [14,16,13] have advanced language model reasoning, yet autoregressive generation still produces arithmetic errors and unfaithful derivations [8,7]. Program-aided approaches delegate computation to external interpreters: PAL [4] and PoT [1] execute numerical Python, MathCoder [12] and ToRA [5] blend tool calls with rationales, and SymCode [10] uses SymPy [9] for exact algebra.

However, execution cannot guarantee mathematical correctness if formalization fails to represent the problem faithfully [8,11]. Most methods synthesize interpretation, planning, and code in a single unconstrained prompt [4,10]. While frontier models manage this joint task, compact models ($\leq 8B$) under 4-bit quantization [3] degrade severely on MATH-500 [7,6] due to four failure patterns:

(1) *semantic misformulation*, solving distorted equations; (2) *target-anchor drift*, diverging from the queried variable or type signature; (3) *domain boundary omission*, admitting extraneous roots without bound checks; and (4) *silent repair regression*, where self-debugging replaces valid solutions with incorrect ones.

To resolve these failures, we introduce **SymPlan**, an inspectable neurosymbolic framework for compact models. SymPlan decouples reasoning into three stages: a bounded *Fused Target-Anchoring Phase* (≤ 256 tokens) fixing the queried variable, formal output signature, and algorithmic roadmap; a *Target-Anchored Code Generator* producing domain-constrained SymPy code; and a *Targeted Debugging Loop* pairing AST diagnostics with an *Anti-Regression Selection* mechanism to preserve valid solutions.

Our contributions are: (1) characterizing formulation, target drift, and repair failures across six compact models on MATH-500; (2) proposing SymPlan to decouple target anchoring, SymPy synthesis, and anti-regression repair; and (3) demonstrating consistent gains over SymCode (+3.60% to +5.20%) and CoT (+12.64%), raising ESR to 89.90% without fine-tuning.

2 Related Work

Program-Aided Reasoning. PAL [4], PoT [1], MathCoder [12], and ToRA [5] offload computation to Python, while SymCode [10] uses SymPy [9] for exact algebra. However, monolithic generation overburdens compact models: misinterpreted equations or omitted domain boundaries yield mathematically invalid code.

Formalization and Code Repair. Faithful CoT [8] separates reasoning from execution, Logic-LM [11] refines symbolic forms, and Self-Debugging [2] patches scripts iteratively. On compact models, unanchored repair causes code cycling and silent regressions. SymPlan grounds repair through an immutable target anchor and anti-regression tracking.

Planning in SLMs. Structured prompting, including Self-consistency [14], Least-to-Most [16], and Plan-and-Solve [13], enhances reasoning in compact models. SymPlan extends these concepts by establishing explicit target anchors and verifiable execution steps tailored for quantized models.

3 Method

3.1 Framework Formulation

Given a problem q , monolithic symbolic approaches [10] generate and execute code in one unconstrained pass: $C_{\text{base}} \sim \mathcal{M}(\cdot \mid q)$, $\hat{y} = \text{Exec}(C_{\text{base}})$. On compact models, this joint generation causes semantic drift and execution crashes. SymPlan structures problem solving into three target-anchored phases:

$$\mathcal{T} = (\tau, \Sigma, \mathcal{P}) = \text{FusedPlan}_{\mathcal{M}}(q), \quad (1)$$

$$C^{(0)} = \text{CodeGen}_{\mathcal{M}}(q, \mathcal{T}), \quad \mathcal{H} = \text{ExecuteAndRepair}_{\mathcal{M}}(C^{(0)}, q, \mathcal{T}), \quad (2)$$

$$\hat{y} = \text{SelectBest}(\mathcal{H}, \Sigma), \quad (3)$$

where τ denotes the queried unknown, Σ is the formal output signature, $\mathcal{P}$ represents the computational plan, and $\mathcal{H}$ records execution history. Like an anchor man stabilizing team structure, establishing $\mathcal{T}$ prior to code generation anchors mathematical intent throughout execution and debugging.

3.2 Fused Target Anchoring and Algorithmic Planning

Multi-turn prompting pipelines often double autoregressive latency and induce context drift in compact models. SymPlan extracts the target anchor and plan $\mathcal{T} = (\tau, \Sigma, \mathcal{P})$ in a single bounded pass (≤ 256 tokens): (1) **Target Specification** (τ): the formal quantity queried (e.g., positive real roots); (2) **Formal Output Signature** (Σ): a discrete type $\Sigma \in \{\text{numeric, text, tuple, set, symbolic}\}$, where cardinality queries map to $\Sigma = \text{numeric}$; and (3) **Computational Plan** ($\mathcal{P}$): ordered symbolic steps $(p_1, \dots, p_m)$ specifying symbols, equations, transformations, and domain boundary checks.

3.3 Target-Anchored SymPy Code Generation

Given $(\tau, \Sigma, \mathcal{P})$, generator $\text{CodeGen}_{\mathcal{M}}$ produces self-contained SymPy code enforcing four invariants: (1) *Exact Arithmetic*: rationals use `sp.Rational(p, q)`; floats are avoided unless requested. (2) *Domain Filtering*: roots from `sp.solve` are checked against domain bounds ($x \in \mathbb{R}^+$, non-zero denominators). (3) *Simplification*: expressions are canonicalized via `sp.together()` or `sp.cancel()`. (4) *Signature Conformance*: the script prints a single boxed L^AT_EX answer strictly conforming to Σ .

```
import sympy as sp; x = sp.Symbol('x', real=True); eq = sp.Eq(sp.sqrt(x), x - 2)
cands = sp.solve((x - 2)**2 - x, x); valid = [s for s in cands if s >= 2 and eq.lhs.subs(x,
s) == eq.rhs.subs(x, s)]
print(r"\boxed{%s}" % sp.latex(valid[0]))
```

3.4 Targeted Debugging with Anti-Regression Selection

Execution proceeds in an isolated sandbox ($\tau = 15\text{s}$, retry limit $K = 2$) with three safeguards: **AST Diagnostics**: Tracebacks $e^{(i)}$ are paired with static AST checks (`lint_sympy_code`) and candidate verification $v^{(i)}$. **Anti-Cycling Repair**: If $C^{(i)} \equiv C^{(i-1)}$, prompt constraints force an alternative implementation strategy. **Anti-Regression Ranking**: Attempts are recorded in $\mathcal{H} = \{r_0, \dots, r_k\}$ and scored via $\text{Rank}(r) = \langle \mathbb{I}_{\text{exec}}(r), \mathbb{I}_{\text{valid}}(r), \mathbb{I}_{\text{verif}}(r), -\text{attempt}(r) \rangle$, prioritizing verified earlier runs over speculative later mutations. The optimal output is $r^* = \arg \max_{r \in \mathcal{H}} \text{Rank}(r)$, with multi-tier regex fallback for malformed outputs.

4 Experiments and Results

Setup. We evaluate on the MATH-500 benchmark [7,6] (7 subjects, 5 difficulty levels) across six open-weight models: Qwen2.5-3B-Instruct, Qwen3-4B,

Table 1. Cross-model accuracy (%) and Execution Success Rate (ESR %) on MATH-500 across six compact models (3B–8B). Best accuracy is in **bold**.

Model	Size	Direct	CoT	SymCode	SymPlan	ESR (SC)	ESR (Ours)
Qwen2.5-3B-Instruct	3B	17.00	39.80	37.40	41.20	80.20%	87.20%
Qwen3-4B	4B	36.00	45.40	56.40	61.60	81.60%	90.20%
Qwen2.5-7B-Instruct	7B	25.40	45.60	54.00	57.60	88.40%	93.80%
Qwen2.5-Coder-7B-Instruct	7B	22.00	38.20	57.00	61.40	89.40%	92.40%
Llama-3.1-8B-Instruct	8B	23.20	37.60	37.60	41.80	76.60%	84.60%
Qwen3-8B	8B	36.20	45.00	59.80	63.80	86.80%	91.20%
Average (6 Models)	All	26.63	41.93	50.37	54.57	83.83%	89.90%

Table 2. Fine-grained accuracy (%) across difficulty levels and subjects on MATH-500 for Qwen2.5-Coder-7B and Qwen3-8B.

Category	Qwen2.5-Coder-7B		Qwen3-8B	
	Count	SymCode	SymPlan	SymCode SymPlan
<i>Difficulty Levels</i>				
Level 1–2	133	78.95	84.96	75.19 81.20
Level 3	105	61.90	73.33	63.81 66.67
Level 4–5	262	43.89	44.66	50.38 53.82
<i>Mathematical Subjects</i>				
Algebra	124	66.94	76.61	71.77 75.81
Counting & Probability	38	52.63	57.89	44.74 57.89
Geometry	41	31.71	34.15	46.34 51.22
Intermediate Algebra	97	51.55	48.45	50.52 51.55
Number Theory	62	66.13	70.97	62.90 72.58
Prealgebra	82	69.51	75.61	73.17 73.17
Precalculus	56	37.50	41.07	46.43 48.21

Qwen2.5-7B-Instruct, Qwen2.5-Coder-7B-Instruct, Llama-3.1-8B-Instruct, and Qwen3-8B. All models run under 4-bit NormalFloat4 (NF4) quantization [3] with greedy decoding (temp = 0.0, max_tokens = 1024). We benchmark against Direct generation, Chain-of-Thought (CoT) [15], and monolithic SymCode [10], measuring Accuracy (% EM) and Execution Success Rate (ESR).

Main Results. Table 1 shows that SymPlan consistently outperforms monolithic SymCode by +3.60% to +5.20% (averaging +4.20%) and CoT by +12.64% (reaching +23.20% on Coder-7B). Average ESR rises from 83.83% to 89.90% (+6.07%), reducing execution failures by 37.5%. In inference cost, SymPlan generates only 22.6% more tokens than SymCode (392.9 vs. 320.6 tokens/problem), remaining substantially shorter than CoT (423.7 tokens).

Stratified Analysis and Ablation. Table 2 shows that gains concentrate in intermediate problems (Level 3: +11.43% on Coder-7B) and subjects with strict algebraic constraints: *Algebra* (+9.67%), *Counting & Prob.* (+13.15%), and *Number Theory* (+9.68%). In component ablations, target anchoring (+1.60% to +2.40%) and algorithmic planning (+2.40% to +3.80%) improve performance

Table 3. Failure mode comparison between SymCode and SymPlan on MATH-500.

Failure Category	SymCode	SymPlan	Net Red.	Rel. Red.
Execution Failures (Syntax/Crash)	485 (16.2%)	303 (10.1%)	-182	-37.5%
Format / Type Mismatches	515 (17.2%)	394 (13.1%)	-121	-23.5%
Semantic Misformulations	974 (32.5%)	969 (32.3%)	-5	-0.5%
Total Incorrect Solutions	1489 (49.6%)	1363 (45.4%)	-126	-8.5%

independently, while combining both with anti-regression ranking attains the highest accuracy (+3.60% to +5.20%).

Error Diagnostics and Mechanistic Insights. As shown in Table 3, SymPlan eliminates 126 errors across 3,000 runs (−8.5% relative drop). Safe SymPy templates lower crashes by 37.5%, and formal output signatures cut format mismatches by 23.5%. In multi-condition algebra, target anchoring registers domain bounds and evaluation points beforehand, preventing constraint omission and enabling exact symbolic reconstruction. Conversely, semantic misformulations remain flat (974 → 969, −0.5%), showing that neurosymbolic scaffolding resolves execution-interface failures while relying on base-model knowledge for non-constructive problem formulation.

5 Conclusion

We presented SymPlan, an inspectable neurosymbolic framework for mathematical reasoning in compact models. By decoupling problem solving into fused target anchoring, specification-guided SymPy synthesis, and targeted debugging with anti-regression ranking, SymPlan resolves boundary omission and repair regressions. Across six quantized models (3B–8B) on MATH-500, SymPlan outperforms monolithic SymCode by +3.60% to +5.20% and CoT by +12.64%, raising execution success to 89.90% without fine-tuning. Future work will extend SymPlan to interactive theorem proving and geometric diagrams.

References

1. Chen, W., et al.: Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. TMLR (2023)
2. Chen, X., et al.: Teaching large language models to self-debug. In: Proc. ICLR (2024)
3. Dettmers, T., et al.: QLoRA: Efficient finetuning of quantized LLMs. In: Proc. NeurIPS. vol. 36, pp. 10088–10115 (2023)
4. Gao, L., et al.: P_{AL}: Program-aided language models. In: Proc. ICML. vol. 202, pp. 10764–10799 (2023)
5. Gou, Z., et al.: ToRA: A tool-integrated reasoning agent for mathematical problem solving. In: Proc. ICLR (2024)
6. Hendrycks, D., et al.: Measuring mathematical problem solving with the MATH dataset. In: Proc. NeurIPS Datasets Track (2021)
7. Lightman, H., et al.: Let’s verify step by step. In: Proc. ICLR (2024)
8. Lyu, Q., et al.: Faithful chain-of-thought reasoning. In: Proc. IJCNLP-AACL. pp. 305–329 (2023)

9. Meurer, A., et al.: SymPy: Symbolic computing in Python. *PeerJ Comput. Sci.* **3**, e103 (2017)
10. Nezhad, S.B., Li, Y., Agrawal, A.: SymCode: A neurosymbolic approach to mathematical reasoning via verifiable code generation. *arXiv:2510.25975* (2025)
11. Pan, L., et al.: Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In: *Proc. EMNLP*. pp. 3806–3824 (2023)
12. Wang, K., et al.: MathCoder: Seamless code integration in large language models for enhanced mathematical reasoning. In: *Proc. ICLR* (2024)
13. Wang, L., et al.: Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In: *Proc. ACL*. pp. 2609–2634 (2023)
14. Wang, X., et al.: Self-consistency improves chain of thought reasoning in language models. In: *Proc. ICLR* (2023)
15. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: *Proc. NeurIPS*. vol. 35, pp. 24824–24837 (2022)
16. Zhou, D., et al.: Least-to-most prompting enables complex reasoning in large language models. In: *Proc. ICLR* (2023)